

TEORÍA (8 puntos)

Apellidos y nombre:

Pregunta 1. Explica en qué consiste una prueba de cobertura de decisiones

Pregunta 2. ¿Qué técnica de pruebas utilizarías para encontrar defectos en el *software* que sean consecuencia de una combinación de una pareja de parámetros?

Pregunta 3. Enumera tres técnicas de depuración de errores

Pregunta 4. Define proceso de pruebas

Pregunta 5. ¿En qué consiste la base de las pruebas (*test basis*)?

Pregunta 6. ¿Quién es el responsable de ejecutar las pruebas?

Pregunta 7. Describe brevemente la tarea de “determinar la estrategia de pruebas” dentro de la elaboración del plan de pruebas maestro

Pregunta 8. Dada la siguiente tabla donde se indica el tamaño de los componentes de un sistema y su clase de riesgo, ¿Cuál sería el número de horas dedicadas a las pruebas de cada componente utilizando una estimación proporcional basada en riesgos si disponemos de 90 horas para dedicar en total al proceso de pruebas y el factor de dedicación para las clases de riesgo A, B y C es 1'5, 1 y 0'5 respectivamente?

Componente	Tamaño	Clase de riesgo	Horas
1	10	A	
2	20	B	
3	20	C	

Pregunta 9. Enumera tres métricas que permitan informar acerca de la calidad del objeto de pruebas, o el progreso y calidad del proceso de pruebas

Pregunta 10. Recomienda tres herramientas/infraestructuras/entornos de automatización de pruebas de sistema o aceptación para aplicaciones en dispositivos Android.

EJERCICIO (2 puntos)**Prueba de caminos con profundidad**

Diseñad pruebas para el siguiente código utilizando la técnica de pruebas de caminos con nivel de profundidad igual a 2:

1. Obtened primero el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/expresiones y nodos que se indica en el propio código. Notad que los nodos G y J corresponden con más de una instrucción (0,25 puntos).
2. Tras haber realizado el grafo de flujo, y a la vista del código, identificad un par de aristas adyacentes al nodo H cuya ejecución consecutiva es imposible y explicad por qué (0,25 puntos).
3. Aplicad la técnica de pruebas de caminos con profundidad 2 para obtener el conjunto de caminos completos correspondiente. No consideréis como situación de prueba el par de aristas de ejecución consecutiva imposible identificado en el punto anterior (0,75 puntos).
4. Finalmente, estableced casos de prueba para los caminos obtenidos, incluyendo entradas y salidas del método (0,75 puntos).

```
/**
 * @param n - un entero cualquiera
 * @return «true» si y solo si «n» es un número primo (es decir,
 *         si es positivo y tiene exactamente dos divisores: 1 y n.
 */
public static boolean esPrimo(int n) {
  A boolean esPrimo;
  B if (n == 2) {
    C esPrimo = true; // «n» es igual a 2, luego es primo
  }
  else if ( D n < 2 || E n % 2 == 0) {
    F esPrimo = false; // «n» es menor que 2 o divisible por 2
  }
  else {
    // Se buscan posibles divisores impares de «n»
    G boolean divisorEncontrado = false;
    int divisor = 3; // Primer divisor impar a probar
    while ( H !divisorEncontrado && I divisor * divisor <= n) {
      J divisorEncontrado = n % divisor == 0;
      divisor = divisor + 2;
    }
    K esPrimo = !divisorEncontrado;
  }
  L return esPrimo;
}
```